

Módulo 1 - Fundamentos, Instalação e Modelo Mental do Git

PDF expandido do curso interativo



Este PDF organiza o conteúdo do curso em formato linear, com descrições de imagem, atividades, opções de interação, feedbacks, acordeões e quiz final expandidos para leitura e impressão.

Sumário

1. Trilha: Git e GitHub
2. Autoria
3. Boas-vindas!
4. Objetivos de aprendizagem
5. Controle de Versão e Colaboração
6. Rastreabilidade, Autoria e Fonte Única de Verdade
7. Evolução dos Sistemas de Controle de Versão
8. História e Motivação do Git
9. Modelo de Dados do Git: Snapshots, Deltas e Banco de Objetos
10. Hash SHA-1, Integridade e Imutabilidade
11. Objetos Internos: Blob, Tree, Commit e Tag
12. O Diretório .git como Banco de Dados do Repositório
13. Grafo Acíclico Direcionado e Histórico Não Linear
14. As Três Árvores: Working Directory, Staging Area e Repository
15. Estados dos Arquivos e Transições
16. Instalação e Preparação do Ambiente
17. Configuração Inicial: Identidade, Escopos e Line Endings
18. Sistema de Ajuda e Leitura de Documentação Técnica
19. Conclusão
20. Referências

Página 1 - Trilha: Git e GitHub

Trilha: Git e GitHub

Trilha: Git e GitHub

Módulo 1 - Fundamentos, Instalação e Modelo Mental do Git



Descrição da imagem: Ilustração sobre fundamentos, instalação e modelo mental do Git

Descrição expandida: A imagem representa visualmente os fundamentos do Git, sua instalação e seu modelo mental, usando ícones de repositório, código, download, configuração, commits, branches e merge para mostrar a ideia de controle de versão, organização do trabalho e evolução segura de um projeto.

Imagem gerada por IA

Menu lateral

Acesse as páginas pelo sumário.

Navegação

Use Anterior, Próximo e dots.

Acordeões

Clique para abrir conteúdos.

Simuladores

Teste estados do Git.

Progresso

O AVA registra sua navegação.

Quiz

Atinja 75% para aprovação.

Página 2 - Autoria

Autoria

Prof. Dr. Eduardo Amadeu Moresi

Prof. Ms. Weldes Lima

Equipe Técnica

Eduardo Amadeu Dutra Moresi

Vilson Hartmann

Mário Braga

Everson Menezes

Isolda Gusmão

Thiago Ribalovsky

Weldes Lima Oliveira

Projeto

Residência em TIC 04B - TIC EM TRILHAS

Direitos Autorais

© 2026 – TIC em Trilhas - Universidade Católica de Brasília UCB

Todos os direitos reservados.

É proibida a reprodução total ou parcial deste material, por qualquer meio, sem autorização expressa da executora.

Sobre o TIC em trilhas

Iniciativa em parceria com TIC em Trilhas e parte integrante do projeto Residência em TIC 04B. Projetos apoiados pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248/91, sendo uma iniciativa do Ministério da Ciência, Tecnologia e Inovações e Softex no âmbito do Programa MCTI Futuro.



Descrição da imagem: Régua de logos do projeto TIC em Trilhas, Católica, Softex, MCTI Futuro e Governo Federal

Descrição expandida: Régua institucional com os logotipos do TIC em Trilhas, Universidade Católica de Brasília, Softex, MCTI Futuro, Ministério da Ciência, Tecnologia e Inovação e Governo Federal.

Identificação institucional do projeto.

Página 3 - Boas-vindas!

Boas-vindas!

Boas-vindas à trilha Git e GitHub!

Você está começando pelo módulo certo. Antes de criar pull requests, revisar contribuições de colegas ou resolver conflitos em equipe, é preciso entender o que acontece dentro do Git quando você registra uma mudança. Este módulo constrói esse fundamento.

Git é um sistema de controle de versão; GitHub é uma plataforma de colaboração construída sobre repositórios Git; e a prática profissional envolve decisões sobre autoria, revisão, rastreabilidade e documentação. Aprender apenas a interface do GitHub sem entender o modelo do Git é como aprender a dirigir sem entender o que é uma marcha.

E nessa trilha você vai aprender de forma progressiva e conseguir usar todo o potencial do Git e do GitHub.

Bons estudos!

Atividade prática

Atividade rápida: diagnostique seu ponto de partida

Marque o que você já consegue fazer antes de iniciar. No final do módulo, volte e compare.

- Sei explicar a diferença entre Git e GitHub.
- Já usei terminal para executar comandos simples.
- Sei o que é um commit.
- Consigo identificar por que copiar pastas não é controle de versão.

Destaque: Registro de aprendizagem: anote qual item ainda parece menos claro.

Página 4 - Objetivos de aprendizagem

Objetivos de aprendizagem

Ao concluir este módulo, você será capaz de:

- Instalar e verificar o Git no seu sistema operacional
- Configurar nome, e-mail, editor padrão e tratamento de quebras de linha
- Inspeccionar um repositório Git, incluindo o diretório.git
- Entender como hashes são gerados e por que eles mudam quando o conteúdo muda
- Registrar commits com mensagens compreensíveis e consultar o histórico

Interação expandida

Antes e depois do controle de versão

Sem Git

Arquivos duplicados, nomes ambíguos, pouca rastreabilidade e recuperação difícil.

Com Git

Histórico auditável, autoria registrada, mensagens explicativas e versões recuperáveis.

Atividade prática

Missão do módulo

Use esta lista como roteiro de prática. Marque cada etapa apenas depois de executá-la ou simular sua execução.

- Verifiquei a instalação com `git --version`.
- Configurei nome e e-mail.
- Localizei o diretório.git.
- Testei alteração de conteúdo e hash.
- Criei um commit com mensagem compreensível.

Página 5 - Controle de Versão e Colaboração

1. Controle de Versão e Colaboração

Pense em um trabalho de graduação desenvolvido ao longo de quatro semanas. Na primeira semana, você cria os arquivos iniciais. Na segunda, corrige erros e reorganiza pastas. Na terceira, testa uma abordagem alternativa. Na quarta, precisa entregar a versão estável, e descobre que sobrescreveu exatamente o trecho que funcionava.

Esse cenário é familiar porque o versionamento manual por cópias de arquivos parece funcionar até que deixa de funcionar. Nomes como projeto-final, projeto-final-v2, projeto-final-agora-vai não preservam raciocínio, não garantem autoria e não permitem reconstruir quando um erro foi introduzido.

O problema não é apenas organizacional: é epistemológico. Sem histórico estruturado, fica difícil demonstrar reprodutibilidade em ciência de dados, justificar uma decisão técnica em engenharia ou apresentar evidências de participação em trabalho colaborativo.

Sistemas de controle de versão (VCS) resolvem esses problemas ao transformar a evolução de um projeto em estrutura auditável. Cada mudança pode ser acompanhada por uma mensagem explicativa. Cada autoria fica associada a uma identidade. Cada estado registrado pode ser recuperado. O projeto ganha memória técnica.

A colaboração é o segundo eixo. Em um repositório versionado, o estudante não apenas envia um arquivo ao professor: apresenta uma sequência de decisões. O colega não recebe apenas o produto final: pode inspecionar o caminho. O orientador não apenas corrige o resultado: pode observar o processo de construção. Pesquisas como as de Fiksel et al. (2019) sobre GitHub Classroom e de Beckman et al. (2020) sobre ensino de estatística mostram ganhos concretos em feedback, acompanhamento e confiança dos estudantes quando o controle de versão é tratado como objetivo de aprendizagem.

Interação expandida

Clique para verificar a rastreabilidade

- Opção/ação: Quem?
 - Feedback: Quem modificou?
- Opção/ação: Quando?
 - Feedback: Quando a alteração foi registrada?
- Opção/ação: Por quê?
 - Feedback: Por qual motivo a mudança foi feita?
- Opção/ação: Efeito?
 - Feedback: Qual efeito teve no projeto?

Atividade prática

Prática: escolha a estratégia mais segura

Você precisa entregar um trabalho em grupo e quer preservar quem alterou o quê. Qual caminho é mais adequado?

- Opção/ação: Criar projeto-final-v8.zip
 - Feedback: Estratégia frágil: nomes de arquivo não registram autoria, motivo nem histórico real.
- Opção/ação: Usar Git com commits descritivos

- Feedback: Correto: o repositório com commits registra autoria, data, mensagem e estado recuperável.
- Opção/ação: Salvar cópia no drive
- Feedback: Parcial: backup ajuda a guardar cópias, mas não organiza histórico de decisões.

Página 6 - Rastreabilidade, Autoria e Fonte Única de Verdade

2. Rastreabilidade, Autoria e Fonte Única de Verdade

Rastreabilidade é a capacidade de reconstruir a trajetória de uma alteração: quem modificou determinado arquivo, quando, por qual motivo e com que efeito sobre o restante do projeto. O Git responde a essas perguntas por meio de commits, metadados de autoria e mensagens.

No Git, a autoria é operacionalizada pela sua configuração de identidade. Cada commit registra nome e e-mail do autor. Por isso, configurar corretamente `user.name` e `user.email` não é detalhe burocrático: é a forma como sua produção será identificada no histórico. Em uma disciplina, commits com identidade errada ou genérica podem comprometer a atribuição de trabalho.

Destaque: A noção de fonte única de verdade merece atenção. Em sistemas distribuídos como o Git, cada clone possui uma cópia completa do repositório, não há dependência técnica de um servidor central para registrar trabalho local. Entretanto, em práticas colaborativas, a equipe elege um repositório remoto (frequentemente no GitHub) como referência compartilhada. Esse repositório atua como ponto de coordenação: onde versões aceitas são integradas, issues são discutidas e releases são publicados. Fonte única de verdade é, portanto, uma convenção social e técnica, não uma limitação do sistema.

Por que isso importa agora? Neste módulo você ainda não vai publicar no GitHub. Mas um repositório local bem organizado, com commits claros, identidade correta e sem arquivos sensíveis, é requisito direto para a colaboração futura. O cuidado local é uma forma de responsabilidade colaborativa antecipada.

Para estudantes de ciência de dados, a rastreabilidade tem dimensão adicional: reprodutibilidade. Blischak, Davenport e Wilson (2016) demonstram que mesmo o uso básico de Git beneficia cientistas ao permitir acompanhar alterações, compartilhar trabalho e recuperar versões específicas de scripts e análises.

Interação expandida

Comparar arquiteturas

Conteúdo de acordeão expandido:

- Sistemas locais:
 - Histórico na própria máquina. Bom para recuperar versões, fraco para colaboração.
- Sistemas centralizados:
 - Servidor autoritativo. Facilita governança, mas cria dependência de servidor.
- Sistemas distribuídos:
 - Cada clone carrega o histórico completo. Permite trabalho offline e redundância.

Atividade prática

Miniestudo de caso: rastreabilidade

Imagine que um arquivo foi alterado e gerou erro. Clique nas perguntas que o histórico do Git deve ajudar a responder.

- Opção/ação: Quem alterou?
 - Feedback: Quem: a autoria vem da configuração `user.name` e `user.email` registrada no commit.
- Opção/ação: Quando alterou?

- Feedback: Quando: o commit registra data e hora do registro histórico.
- Opção/ação: Por que alterou?
 - Feedback: Por quê: a mensagem de commit deve explicar a intenção da mudança.
- Opção/ação: O que mudou?
 - Feedback: Efeito: o diff permite comparar o que mudou entre versões.

Página 7 - Evolução dos Sistemas de Controle de Versão

3. Evolução dos Sistemas de Controle de Versão

A história dos sistemas de controle de versão pode ser contada como uma sequência de respostas a problemas reais de coordenação, redundância e autonomia. Cada geração resolve dificuldades anteriores e introduz novos desafios.

Sistemas locais

Na forma mais simples de controle estruturado, o histórico fica armazenado na própria máquina do usuário. Ferramentas como RCS mantinham registros de diferenças entre versões em banco local. A vantagem era clara: permitia recuperar estados anteriores sem depender de cópias manuais. A limitação também: colaboração entre múltiplos autores continuava sendo problema externo ao sistema.

Sistemas centralizados

CVS, Subversion e Perforce responderam à necessidade de colaboração com um servidor central que armazena o repositório autoritativo. A centralização simplifica permissões e visibilidade administrativa. O problema é a dependência: se o servidor estiver indisponível, commits ficam bloqueados; se ele sofrer falha sem backup, o histórico pode ser perdido.

Sistemas distribuídos

Git, Mercurial e Bazaar alteraram a lógica. Ao clonar um repositório, você não obtém apenas a versão mais recente dos arquivos, obtém uma cópia completa do histórico. Isso permite trabalho offline, commits locais, redundância natural e fluxos hierárquicos de integração. O repositório remoto continua importante para colaboração, mas não é condição para que você registre seu progresso local.

Para estudantes em curso autoinstrucional, essa característica é especialmente relevante: você pode experimentar, errar, recuperar e refletir na sua própria máquina, sem depender de infraestrutura remota durante o aprendizado.

Arquitetura	Característica principal	Vantagem para aprendizagem	Limitação a discutir
Local	Histórico armazenado na máquina do usuário	Mostra a ideia básica de registrar versões	Não resolve colaboração entre autores
Centralizada	Servidor único armazena histórico autoritativo	Facilita governança e controle de acesso	Dependência do servidor; ponto único de falha
Distribuída	Cada clone contém histórico completo	Permite trabalho offline e experimentação segura	Exige compreensão de histórico local e sincronização

Interação expandida

Princípios do Git

Velocidade

Não linear

Integridade

Descentralização

Atividade prática

Atividade: relacione arquitetura e consequência

Clique em cada arquitetura e observe a consequência pedagógica.

- Opção/ação: Sistema local
 - Feedback: Local: bom para entender versões, mas não resolve colaboração entre autores.
- Opção/ação: Sistema centralizado
 - Feedback: Centralizado: simplifica controle, mas depende do servidor central.
- Opção/ação: Sistema distribuído
 - Feedback: Distribuído: cada clone contém histórico completo, favorecendo autonomia e trabalho offline.

Página 8 - História e Motivação do Git

4. História e Motivação do Git

O Git não nasceu como ferramenta genérica de produtividade. Ele foi criado em 2005 por Linus Torvalds como resposta a uma crise real: o projeto Linux precisava substituir rapidamente o BitKeeper, um sistema distribuído proprietário cujo uso gratuito havia sido revogado por conflitos de licenciamento.

O Linux envolvia milhares de contribuidores geograficamente dispersos. O novo sistema precisava ser rápido, suportar desenvolvimento não linear, proteger a integridade dos dados e eliminar dependência de nó central. Esses princípios não são curiosidades históricas, eles explicam por que o Git funciona como funciona.

Quando você aprende que um branch é leve, que um commit é identificado por hash, que o histórico forma um grafo e que a maior parte das operações é local, você está vendo esses princípios originais em ação. O comando `git commit` não é apenas uma instrução para salvar: ele cria um objeto em um banco de dados local. O comando `git branch` não duplica o projeto: cria uma referência leve para um commit. Entender a origem do Git transforma comandos em conceitos.

Princípios de projeto do Git
Velocidade → operações são locais e não dependem de rede.
Desenvolvimento não linear → branches são leves e merges são eficientes. **Integridade** → objetos são identificados por hash; qualquer alteração é detectável. **Descentralização** → cada clone contém histórico completo.

Interação expandida

Simulador de snapshot

Arraste mentalmente a ideia: o Git registra o estado do projeto, não apenas uma lista de linhas.

Snapshot A

Snapshot B

Snapshot C

Atividade prática

Prática conceitual: princípio → comando

Associe cada princípio original do Git a uma ação concreta.

Conteúdo de acordeão expandido:

- Velocidade:
 - Prática: execute operações locais como `git status` e `git log` sem depender de rede.
- Integridade:
 - Prática: altere um caractere e observe que o hash muda completamente.
- Descentralização:
 - Prática: clone um repositório e perceba que o histórico acompanha a cópia local.
- Desenvolvimento não linear:
 - Prática futura: branches permitirão trabalhar em linhas diferentes sem copiar pastas.

Página 9 - Modelo de Dados do Git: Snapshots, Deltas e Banco de Objetos

5. Modelo de Dados do Git: Snapshots, Deltas e Banco de Objetos

Aqui está uma das ideias mais importantes do módulo: o Git não funciona guardando listas de linhas adicionadas e removidas. Ele registra estados completos do projeto.

A cada commit, o Git captura um instantâneo lógico, um snapshot, do conjunto de arquivos naquele momento. Se um arquivo não mudou, o Git não duplica o conteúdo: reutiliza a referência ao objeto já existente. Eficiência e histórico completo coexistem.

Por baixo dos panos, cada conteúdo armazenado se torna um objeto chamado blob, identificado por um hash. Estruturas de diretório são representadas por trees, que apontam para blobs e outras trees. Um commit aponta para uma tree raiz e contém metadados: autor, data, mensagem e referência ao commit pai. Essa organização permite reconstruir estados completos sem que cada commit armazene tudo de forma redundante.

O Git funciona, portanto, como um banco de dados de objetos endereçável por conteúdo. A identidade de um objeto deriva do seu conteúdo, não de um contador sequencial. Se o conteúdo muda, o hash muda, e isso sustenta a integridade do histórico.

Cuidado com simplificações "Git é backup." Essa metáfora ajuda no início, mas é incompleta. Backup preserva cópias; Git preserva história estruturada com autoria, mensagens e relações entre estados. "Git é sincronização." Também parcial: a sincronização com GitHub só faz sentido quando o histórico local está bem construído.

Interação expandida

Simulador de hash

Digite um conteúdo e veja um identificador simulado mudar quando o texto muda.

Atividade prática

Simulação: snapshot ou delta?

Situação: em um commit, apenas um arquivo mudou. O Git precisa duplicar todo o projeto?

- Opção/ação: Não, reutiliza objetos existentes
 - Feedback: Correto. O Git registra o estado lógico do projeto e reutiliza referências para objetos que não mudaram.
- Opção/ação: Sim, duplica todos os arquivos
 - Feedback: Incorreto. Snapshot não significa duplicar fisicamente tudo a cada commit.
- Opção/ação: Guarda só uma lista de linhas
 - Feedback: Parcial. O modelo mental apresentado é de snapshots, não de simples lista de linhas.

Painel de resultado: Escolha uma resposta.

Página 10 - Hash SHA-1, Integridade e Imutabilidade

6. Hash SHA-1, Integridade e Imutabilidade

O Git identifica cada objeto por um hash SHA-1: uma sequência hexadecimal de 40 caracteres calculada a partir do conteúdo do objeto. Dois conteúdos idênticos geram o mesmo hash. Uma alteração mínima, um caractere, um espaço, gera um hash completamente diferente.

Essa propriedade tem uma consequência fundamental: commits são imutáveis. Não porque seja impossível reescrever a história com comandos avançados, mas porque alterar qualquer elemento de um commit, mensagem, autor, data, conteúdo, cria um novo objeto com um novo hash. O commit original permanece inalterado no banco de objetos até ser coletado pelo garbage collector.

Para você como estudante, isso significa que o histórico do Git é confiável por construção. Alterações silenciosas são detectáveis. Em contextos acadêmicos e científicos, essa propriedade permite demonstrar que determinado arquivo estava em certo estado no momento de um commit, o que é relevante tanto para avaliação quanto para reprodutibilidade.

Destaque: Há também uma implicação prática importante: o que entra no histórico tende a permanecer recuperável. Por isso, revisar o que você está commitando é hábito fundamental. Senhas, tokens de acesso, dados pessoais e arquivos temporários não devem entrar no repositório. O módulo 2 aprofundará o uso do .gitignore para prevenir isso.

Experimento com hash, prévia do Laboratório 2

Crie um arquivo com o conteúdo "O Git é um grafo" e execute `git hash-object`. Anote o hash. Adicione um ponto de exclamação e repita o comando, o hash muda completamente. Volte ao conteúdo original e o hash inicial reaparece. Esse experimento simples demonstra tudo o que foi explicado nesta seção.

Interação expandida

Hotspots do diretório.git

- Opção/ação: `objects/`
 - Feedback: `objects/`: banco de dados dos objetos do Git.
- Opção/ação: `refs/`
 - Feedback: `refs/`: referências como branches e tags.
- Opção/ação: `HEAD`
 - Feedback: `HEAD`: ponteiro para o estado ativo.
- Opção/ação: `config`
 - Feedback: `config`: configurações locais.
- Opção/ação: `index`
 - Feedback: `index`: área de preparação interna.

Atividade prática

Laboratório simulado: teste o hash

Digite um texto, copie o hash gerado, altere apenas um caractere e compare.

Destaque: Tarefa: explique em uma frase por que uma pequena mudança altera o identificador.

Página 11 - Objetos Internos: Blob, Tree, Commit e Tag

7. Objetos Internos: Blob, Tree, Commit e Tag

O Git trabalha com quatro tipos de objetos. Você não precisa manipulá-los diretamente no dia a dia, mas compreendê-los transforma a ferramenta de caixa-preta em sistema coerente.

Objeto	O que armazena	Pergunta que responde	Comando para explorar
Blob	Conteúdo de um arquivo (sem nome ou caminho)	Quais bytes compõem este conteúdo?	git hash-object; git cat-file -p
Tree	Estrutura de diretório: nomes, permissões e hashes	Onde o conteúdo aparece e com qual nome?	git cat-file -p <hash da tree>
Commit	Snapshot histórico: aponta para tree + metadados + pai	Que estado foi salvo e qual é seu ancestral?	git log; git cat-file -p <hash>
Tag	Referência nomeada para um commit importante	Que commit representa uma versão estável?	git tag

Um detalhe importante sobre o blob: ele armazena apenas conteúdo, sem nome de arquivo ou caminho. Isso significa que o mesmo conteúdo presente em arquivos com nomes diferentes gera um único blob. A tree é quem resolve o mapeamento entre conteúdo e nome.

A tag pode ser apresentada aqui de forma conceitual: é uma referência nomeada, frequentemente usada para marcar versões como v1.0.0. O que importa agora é perceber que o Git diferencia objetos (armazenados no banco) de referências (ponteiros para objetos). Essa distinção será essencial para entender branches no Módulo 2.

Interação expandida

Visualizador de DAG

A
B
C
D
E

O histórico pode se bifurcar e se reunir sem virar “cópia de pasta”.

Atividade prática

Flashcards de prática: objetos internos

Blob

Armazena o conteúdo do arquivo, sem nome nem caminho.

Tree

Mapeia nomes, permissões e caminhos para blobs e outras trees.

Commit

Aponta para uma tree, registra metadados e referencia o commit pai.

Tag

Referência nomeada para um commit importante, como uma versão estável.

Página 12 - O Diretório.git como Banco de Dados do Repositório

8. O Diretório.git como Banco de Dados do Repositório

Quando você executa `git init` em uma pasta, o Git cria o diretório oculto `.git`. Esse diretório é o repositório propriamente dito. Os arquivos que você edita compõem o diretório de trabalho; a memória histórica, as configurações locais, os objetos e as referências ficam dentro de `.git`.

É fundamental desfazer uma ideia comum: o repositório não é apenas a pasta com os arquivos do projeto. Ela contém duas dimensões: a área visível de trabalho e a infraestrutura oculta de versionamento.

O que você vai encontrar dentro de `.git`

- `objects/`, banco de dados com todos os objetos (blobs, trees, commits, tags)
- `refs/`, referências como branches e tags
- `HEAD`, indica qual branch ou commit está ativo no momento
- `config`, configurações locais do repositório
- `index`, representação interna da área de preparação (staging area)

Atenção Não edite manualmente os arquivos dentro de `.git` durante as atividades introdutórias. A finalidade aqui é observacional: olhar para a estrutura, relacionar comandos a efeitos e desenvolver curiosidade técnica responsável.

Explorar o `.git` também explica por que copiar apenas os arquivos do projeto não preserva histórico. Se você copia a pasta sem `.git`, leva o estado atual dos arquivos, mas perde todos os commits, referências e objetos. Por isso, a entrega ideal de um trabalho versionado não é um arquivo compactado sem histórico, mas um repositório com evidências.

Interação expandida

Simulador das três árvores

Working Directory

arquivo.md modificado

Staging Area

vazio

Repository

último commit

- Opção/ação: `git add`
- Opção/ação: `git commit`
- Opção/ação: editar novamente

Atividade prática

Prática guiada: investigue o diretório `.git`

Clique nos itens e registre no relatório o papel de cada componente.

- Opção/ação: `objects/`
 - Feedback: `objects/`: banco de dados com blobs, trees, commits e tags.
- Opção/ação: `refs/`

- Feedback: refs/: referências como branches e tags.
- Opção/ação: HEAD
 - Feedback: HEAD: aponta para o estado ativo da branch ou commit.
- Opção/ação: config
 - Feedback: config: configurações locais do repositório.
- Opção/ação: index
 - Feedback: index: representação interna da staging area.

Destaque: Tarefa: observe, mas não edite manualmente os arquivos dentro de .git.

Página 13 - Grafo Acíclico Direcionado e Histórico Não Linear

9. Grafo Acíclico Direcionado e Histórico Não Linear

O histórico do Git é um Grafo Acíclico Direcionado, DAG. O nome parece técnico, mas a ideia é direta: cada commit aponta para seu commit pai, formando uma cadeia histórica. O grafo é direcionado porque os ponteiros têm sentido (do mais recente para o mais antigo). É acíclico porque, seguindo as relações de ancestralidade, você nunca volta ao ponto de partida.

Neste módulo, seu histórico será linear: commit A, depois B, depois C. Mesmo assim, o Git não está armazenando uma lista: está criando objetos em que cada commit referencia o anterior. Quando branches forem introduzidos no Módulo 2, o histórico poderá se bifurcar e se reunir. Compreender o DAG agora evita que você interprete branches mais tarde como "cópias de pastas".

O DAG também explica a eficiência do Git. Para recuperar um commit específico, o sistema identifica a tree associada e reconstrói o conteúdo a partir dos objetos correspondentes. Para comparar versões, percorre relações no grafo. Para determinar ancestralidade, avalia caminhos entre commits.

Visualizando o histórico Após criar alguns commits, execute: `git log --oneline` `git log --oneline --graph` Veja como cada linha representa um commit e como as setas de parentesco aparecem no segundo comando. Essa visualização é sua janela para o grafo.

Interação expandida

Ciclo dos estados

- Opção/ação: Avançar estado

Painel de resultado: untracked: arquivo existe, mas o Git ainda não acompanha.

Atividade prática

Atividade: leia o histórico como grafo

Observe o exemplo e responda: quais commits são ancestrais de C?

A

B

C

D

- Opção/ação: Ver resposta
 - Feedback: C aponta para B, e B aponta para A. Portanto, B e A são ancestrais de C.

Página 14 - As Três Árvores: Working Directory, Staging Area e Repository

10. As Três Árvores: Working Directory, Staging Area e Repository

O modelo das três árvores é a chave prática deste módulo. Ele explica por que `git add` existe, por que nem tudo que você editou entra automaticamente no commit e por que um arquivo pode estar em dois estados ao mesmo tempo.

Working Directory, a camada visível

É onde você trabalha: abre o editor, cria arquivos, modifica código, escreve Markdown. Aqui, alterações existem sem que o Git as registre no histórico. Um arquivo novo é `untracked` (desconhecido pelo Git). Um arquivo já versionado que foi editado é `modified`. O diretório de trabalho é sua área de experimentação.

Staging Area (Index), a camada de curadoria

Ao executar `git add`, você seleciona uma versão de um arquivo para o próximo commit. Essa seleção é mais precisa do que simplesmente "marcar um arquivo": o Git registra o estado do conteúdo naquele momento. Por isso, um arquivo pode aparecer simultaneamente como `staged` e `modified`, a versão no índice foi preparada, mas você fez mais alterações no diretório de trabalho depois. Isso demonstra que o Git trabalha com snapshots, não apenas com nomes de arquivos.

Repository (Commit History), a camada persistente

Ao executar `git commit`, o Git pega o que está no índice e cria um novo commit no repositório local. Alterações que ficaram apenas no Working Directory não entram. Alterações preparadas entram conforme estavam quando foram adicionadas ao índice. Isso permite commits atômicos e seletivos: você pode editar vários arquivos, mas escolher registrar apenas a correção de um erro, deixando uma funcionalidade experimental para depois.

O papel do HEAD

- HEAD é uma referência para o estado atualmente ativo, em geral, o último commit da branch atual. Quando `git status` informa alterações, ele compara o diretório de trabalho e o índice com o estado do HEAD. Entender o HEAD agora prepara a compreensão futura de `checkout`, `switch` e o estado chamado "detached HEAD".

Camada	Descrição	Comando principal	Erro comum de iniciante
Working Directory	Arquivos visíveis e editáveis no disco	Edição manual; <code>git status</code>	Acreditar que editar arquivo já cria versão no histórico
Staging Area / Index	Seleção do próximo snapshot	<code>git add</code> ; <code>git restore --staged</code>	Acreditar que <code>git add</code> já registra um commit definitivo
Repository	Histórico persistente de commits	<code>git commit</code> ; <code>git log</code>	Acreditar que todo arquivo editado entra automaticamente no commit

Interação expandida

Checklist de instalação

- Git instalado
- Versão conferida com `git --version`
- Editor padrão definido
- Quebras de linha configuradas

Atividade prática

Simulador: execute o fluxo das três árvores

Siga a sequência: editar → adicionar → editar novamente → commitar. Observe por que um arquivo pode aparecer como staged e modified.

Working Directory

limpo

Staging Area

vazio

Repository

último commit

- Opção/ação: 1. Editar arquivo
- Opção/ação: 2. `git add`
- Opção/ação: 3. Editar de novo
- Opção/ação: 4. `git commit`
- Opção/ação: Reiniciar

Painel de resultado: Comece editando o arquivo.

Página 15 - Estados dos Arquivos e Transições

11. Estados dos Arquivos e Transições

Saber ler git status é tão importante quanto saber executar git add ou git commit. Os quatro estados básicos orientam todas as suas decisões no fluxo de trabalho.

- **Untracked:** arquivo existe no diretório de trabalho, mas o Git ainda não o acompanha. Ele pode ser um arquivo temporário, uma saída de compilação ou algo que você ainda decide se vai versionar.
- **Modified:** arquivo já conhecido pelo Git que sofreu alterações no diretório de trabalho. O comando git diff mostra exatamente o que mudou, linha a linha.
- **Staged:** versão do arquivo preparada para o próximo commit. A staging area é sua área de curadoria, você decide o que vai compor a próxima unidade histórica.
- **Committed:** conteúdo armazenado com segurança no repositório local dentro de um commit. Atenção: "seguro localmente" não significa enviado ao GitHub, isso acontece no Módulo 2.

O ciclo de transições pode ser praticado desta forma:

Criar arquivo → untracked

git add → staged

git commit → committed

Editar novamente → modified

git add → staged novamente (nova versão)

git restore --staged → devolve a modified sem perder alterações

Ao praticar esse ciclo conscientemente, você desenvolve fluência operacional e reduz o risco de comandos destrutivos.

Interação expandida

Gerador de comandos de configuração

- Opção/ação: Gerar comandos

Atividade prática

Diagnóstico de estados

Escolha o estado que corresponde à situação.

Conteúdo de acordeão expandido:

- Arquivo criado, mas nunca adicionado ao Git:
 - **Untracked.** O Git ainda não acompanha esse arquivo.
- Arquivo conhecido pelo Git foi editado:
 - **Modified.** O conteúdo mudou no diretório de trabalho.
- Arquivo foi preparado com git add:
 - **Staged.** A versão está pronta para o próximo commit.
- Conteúdo entrou em um commit local:
 - **Committed.** O snapshot foi registrado no repositório.

Página 16 - Instalação e Preparação do Ambiente

12. Instalação e Preparação do Ambiente

A instalação do Git faz parte do conteúdo de aprendizagem, não é uma etapa administrativa que você resolve antes de começar. Configurações incorretas geram problemas que aparecem mais tarde, especialmente em ambientes Windows.

Windows

Destaque: Use o Git for Windows, disponível em git-scm.com. Durante a instalação, preste atenção a:

Editor padrão: escolha um editor que você conhece, como Visual Studio Code.

PATH: habilite o uso do Git pela linha de comando e por softwares de terceiros.

Line endings: escolha "Checkout Windows-style, commit Unix-style line endings" para evitar problemas de colaboração.

Terminal: Git Bash aproxima a sintaxe de ambientes Unix e facilita seguir instruções padronizadas.

macOS

Instale pelas Xcode Command Line Tools (`xcode-select --install`) ou pelo Homebrew (`brew install git`).

Linux

Use o gerenciador de pacotes da sua distribuição: `apt install git`, `dnf install git` ou equivalente.

Verificação

Em qualquer sistema, execute:

```
git --version
```

Esse comando simples confirma que a instalação está acessível pelo terminal. Anote a versão, ela fará parte do seu relatório como evidência de ambiente.

Por que linha de comando? Git é primariamente uma ferramenta de terminal. Interfaces gráficas ajudam, mas o terminal oferece precisão, documentação abundante e reprodutibilidade. Quando você entende os comandos, qualquer interface gráfica se torna mais fácil de usar, e não o contrário.

Interação expandida

Mini-guia de leitura da documentação

O que significam []?

Argumentos opcionais.

O que significam < >?

Variáveis que você substitui pelo valor real.

Como sair do manual?

Pressione q.

Atividade prática

Checklist operacional de instalação

Use este bloco durante a prática no seu computador.

- Instalei o Git pelo caminho indicado para meu sistema operacional.
- Abri o terminal ou Git Bash.
- Executei git --version.
- Anotei a versão para usar como evidência no relatório.

```
$ git --version
```

```
git version 2.x.x
```

Página 17 - Configuração Inicial: Identidade, Escopos e Line Endings

13. Configuração Inicial: Identidade, Escopos e Line Endings

Após instalar o Git, você precisa configurar sua identidade. Essas informações serão incorporadas a cada commit que você fizer:

```
git config --global user.name "Seu Nome Completo" git config --global user.email "seu@email.com"
```

Use um nome reconhecível e um e-mail que esteja associado à sua conta GitHub (ou institucional, conforme orientação da disciplina). Isso garante que sua produção será corretamente identificada no histórico.

Escopos de configuração

O Git possui três níveis de configuração, em ordem crescente de prioridade:

system: aplica-se a todos os usuários da máquina.

global: aplica-se ao seu usuário do sistema operacional. É onde você define identidade e preferências gerais.

local: aplica-se apenas ao repositório atual. Tem prioridade sobre as demais, importante para laboratórios de configuração.

Para ver todas as configurações e suas origens:

```
git config --list --show-origin
```

Tratamento de quebras de linha

Windows usa CRLF para indicar fim de linha; Linux e macOS usam LF. Se essa diferença não for normalizada, o Git pode interpretar que várias linhas foram alteradas quando só houve conversão de formato, gerando ruído no histórico.

Windows: `git config --global core.autocrlf true`

macOS e Linux: `git config --global core.autocrlf input`

Configuração e ética Um commit com identidade errada compromete atribuição de autoria. Um e-mail pessoal exposto em repositório público tem implicações de privacidade. Um nome genérico dificulta o feedback do professor. Configurar corretamente não é formalidade: é o início da responsabilidade profissional no versionamento.

Interação expandida

Checklist de conclusão

- Entendo Git como histórico de snapshots
- Sei diferenciar Working Directory, Staging Area e Repository
- Consigo explicar hash, commit e DAG
- Sei consultar ajuda com `git help`

Atividade prática

Gerador de comandos de configuração

Preencha os campos para gerar os comandos.

Laboratório rápido: line endings

O comando aparecerá aqui.

Página 18 - Sistema de Ajuda e Leitura de Documentação Técnica

14. Sistema de Ajuda e Leitura de Documentação Técnica

Nenhum módulo de 10 horas vai ensinar todos os comandos do Git. O objetivo é formar um usuário capaz de interpretar estados e buscar a operação adequada de forma autônoma. Para isso, você precisa saber usar a documentação oficial.

O Git oferece três formas principais de ajuda:

```
git help <comando> git <comando> --help man git-<comando>
```

Todas abrem a documentação detalhada do comando no terminal. Para navegar: use as setas ou espaço para rolar, / para pesquisar, n e N para avançar entre resultados e q para sair.

Lendo a documentação com eficiência

A documentação do Git usa convenções de sintaxe que precisam ser entendidas:

- Colchetes [] indicam argumentos opcionais.
- Chaves angulares < > indicam variáveis a substituir. Em git add <arquivo>, você substitui <arquivo> pelo nome real, sem os sinais de menor e maior.
- Barras verticais | indicam alternativas.
- Reticências... indicam repetição ou lista.

Incorpore o uso de ajuda aos exercícios. Em vez de receber todos os comandos prontos, consulte git help status antes de usá-lo. Procure a opção que exibe histórico resumido em git help log. Identifique a diferença entre --global e --local em git help config. Essas tarefas desenvolvem a alfabetização técnica que qualquer profissional de tecnologia precisa.

Atividade prática

Prática de autonomia: onde buscar ajuda?

Digite um comando Git e gere a consulta de documentação correspondente.

```
git help <comando>
```

Destaque: Tarefa: use a documentação para descobrir uma opção útil de git log.

Página 19 - Conclusão

15. Conclusão

Este módulo apresentou o Git como sistema distribuído de controle de versão que organiza a evolução de projetos por meio de snapshots, objetos, hashes e relações de ancestralidade.

Você partiu de um problema concreto, o versionamento manual por cópias de pastas, e compreendeu como sistemas locais, centralizados e distribuídos respondem a demandas crescentes de autonomia, redundância e colaboração. A história do Git revelou princípios de projeto que explicam por que o sistema funciona como funciona.

O modelo de dados baseado em snapshots mostrou que o Git não guarda apenas listas de linhas adicionadas e removidas: registra estados completos por meio de objetos (blobs, trees, commits, tags) identificados por hashes. O DAG organizou o histórico como estrutura de dependências entre estados, preparando o terreno para branches e merges.

A mecânica das três árvores ofereceu o modelo operacional do dia a dia: Working Directory, Staging Area e Repository representam camadas distintas, e os estados untracked, modified, staged e committed são sinais para decisão, não apenas rótulos.

O produto de aprendizagem, o repositório laboratorio-git, transforma ação técnica em evidência reflexiva. Ao produzir relatórios em Markdown e organizar commits progressivos, você praticou o que os módulos seguintes vão ampliar: trabalho com remotes, branches, pull requests e colaboração no GitHub.

Atividade prática

Entrega prática do módulo

Antes de avançar para o Módulo 2, confirme se você produziu evidências mínimas.

- Repositório local criado com git init.
- Pelo menos dois commits com mensagens claras.
- Relato curto sobre hash e imutabilidade.
- Explicação própria das três árvores.
- Consulta feita com git help.

Destaque: Critério de prontidão: você consegue olhar para git status e decidir o próximo passo.

Página 20 - Referências

Referências

CHACON, Scott; STRAUB, Ben. Pro Git. 2. ed. Tradução para português. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 18 ago. 2024.

CÓDIGO FONTE TV. Code Review no GitHub. YouTube, 2023. Disponível em: https://www.youtube.com/watch?v=2alg7MQ6_sl. Acesso em: 18 ago. 2024.

DUDLER, Roger. Git - Guia prático. 2023. Disponível em: https://rogerdudler.github.io/git-guide/index.pt_BR.html. Acesso em: 18 ago. 2024.

GITHUB. GitHub Docs. Disponível em: <https://docs.github.com/>. Acesso em: 18 ago. 2024.

Quiz final

Critério do curso: atinja 75% para aprovação.

1. Git é:

- Um sistema de controle de versão (correta)
- Uma rede social
- Um editor de texto
- Um navegador

2. GitHub é:

- Um serviço/plataforma para repositórios Git (correta)
- O diretório.git
- Um comando de terminal
- Um tipo de hash

3. O que git add faz?

- Prepara conteúdo para commit (correta)
- Publica no GitHub
- Apaga histórico
- Cria branch remota

4. O commit registra:

- Um snapshot preparado (correta)
- Apenas arquivos abertos
- Somente nomes de pasta
- A senha do usuário

5. Hash muda quando:

- O conteúdo muda (correta)
- O computador reinicia
- A tela muda
- O mouse clica

6. Working Directory é:

- Área visível de edição (correta)
- Histórico remoto
- Servidor GitHub
- Manual do Git

7. Staging Area serve para:

- Curadoria do próximo snapshot (correta)
- Baixar arquivos
- Criar conta
- Editar imagem

8. git help serve para:

- Consultar documentação (correta)
- Enviar commits
- Excluir repositório
- Criar senha

Glossário

Git: Sistema distribuído de controle de versão.

GitHub: Plataforma de hospedagem e colaboração baseada em repositórios Git.

Commit: Registro histórico de um snapshot preparado.

Snapshot: Estado lógico do projeto em determinado momento.

Hash: Identificador calculado a partir do conteúdo.

DAG: Grafo Acíclico Direcionado usado para representar histórico.

Working Directory: Área visível onde os arquivos são editados.

Staging Area: Área de preparação do próximo commit.

Repository: Histórico persistente de commits.

HEAD: Referência para o estado ativo.

Blob: Objeto Git que armazena conteúdo de arquivo.

Tree: Objeto Git que mapeia nomes e caminhos para objetos.